02

Recap: HTML und CSS

Bedeutung · Kaskade · Box-Modell · Layout · Responsive
Web-Applikationen · Bachelor Informatik · HTWG Konstanz · WS 2026/27 ·
MDN Learn Web Development: HTML und CSS

Lernziele

- ✓ Erklären, warum semantisches HTML für Screenreader, Suchmaschinen und euer Team zählt, und eine Seite aus lauter `div`s umbauen.
- ✓ Ein Formular zugänglich bauen und begründen, warum Browser-Validierung keine Sicherheit ist.
- ✓ Vorhersagen, welche CSS-Regel greift (Kaskade, Spezifität, Vererbung).
- ✓ Das Box-Modell erklären und `box-sizing: border-box` begründen.
- ✓ Zwischen Flexbox und Grid wählen und ein Layout nach Mobile-First bauen.

Worum es heute geht

HTML und CSS kennt ihr. Ihr habt beides schon geschrieben, vielleicht in einem früheren Kurs, vielleicht in einem eigenen Projekt. Deshalb ist dieses Kapitel kein Nachschlagewerk aller Elemente und Eigenschaften, sondern ein **Recap mit Fokus**: Wir schauen uns die wenigen Konzepte an, die in der Praxis immer wieder schiefgehen und die ihr für das ganze Semester braucht.

Dafür begleitet uns ein einziges Beispiel durch das ganze Kapitel: die Seite für einen **Campus Hackathon**. Sie hat eine Kopfzeile mit Navigation, einen Titel mit Datum, ein Bild, ein Programm aus drei Punkten und ein Anmeldeformular. Diese Seite gibt es in drei Stufen, die ihr im Ordner `beispiel/` findet und im Browser öffnen könnt:

Stufe	Datei	Was sich ändert
1	`01_div_suppe.html`	Sieht okay aus, ist aber fast nur aus `div` und Inline-Styles gebaut
2	`02_semantisch.html`	Gleicher Inhalt, jetzt mit Bedeutung. Noch ganz ohne CSS
3	`03_gestylt.html` + `styles.css`	Gleiches HTML wie Stufe 2, dazu ein sauberes Stylesheet

Warum das auch in Zeiten von React wichtig ist: Eine React-Komponente erzeugt am Ende genau dieses HTML. JSX ist HTML mit ein paar Abweichungen, und jedes Styling-System von Tailwind bis CSS Modules erzeugt am Ende CSS, das den Regeln der Kaskade folgt. Wer die Grundlagen versteht, versteht auch, was die Frameworks tun.

Das Problem: die Div-Suppe

Stufe 1 sieht auf dem Desktop ordentlich aus. Blaue Kopfzeile, großer Titel, drei Programmkarten, ein Formular. Wer nur auf den Bildschirm schaut, findet keinen Fehler.

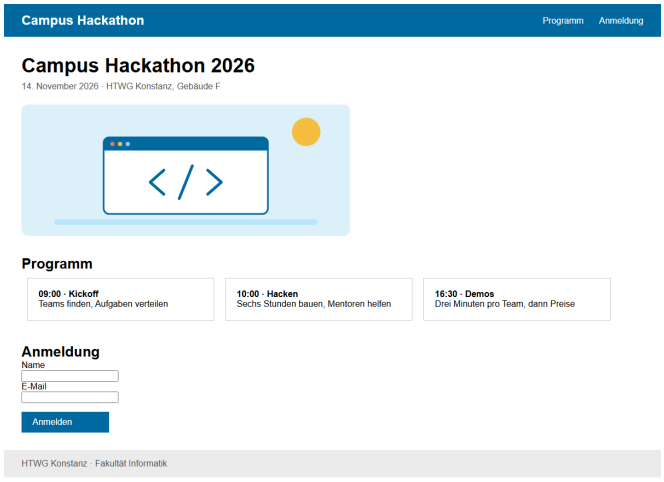


Abbildung 1: Stufe 1 auf dem Desktop

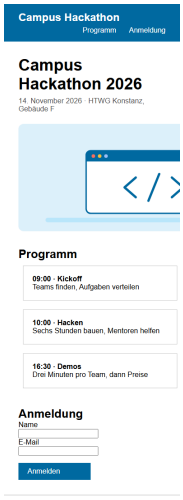


Abbildung 2: Stufe 1 auf dem Handy

Das Problem zeigt sich, sobald jemand anderes als ein Mensch mit Maus und großem Bildschirm auf die Seite schaut. Eine Webseite hat viele Leser:

Wer liest mit?	Was bei der Div-Suppe passiert
Screenreader	Liest nur Text vor. Es gibt keine Navigation, keinen Hauptbereich, keine Überschriften, zu denen man springen könnte.
Tastatur	Die „Links“ und der „Button“ sind `span` und `div` mit `onclick`. Mit der Tab-Taste erreicht man sie nicht.
Suchmaschine	Erkennt nicht, was Titel, was Datum und was Navigation ist.
Handy	Feste Breiten (`width: 600`, `width: 300px` plus Padding) laufen über den Rand. Ohne Viewport-Angabe wird die Seite verkleinert dargestellt.
Formular	Ohne `label` weiß niemand, welches Feld wofür ist. Ohne `name` schickt das Formular nichts ab.
Euer Team	Inline-Styles an 20 Stellen. Wer die Farbe ändern will, sucht lange.

★ HTML beschreibt, **was** etwas ist. CSS beschreibt, **wie** es aussieht. Die Div-Suppe vermischt beides und lässt die Bedeutung weg. Deshalb sieht sie gut aus und funktioniert trotzdem schlecht.

HTML beschreibt Bedeutung

Das Grundgerüst

Jede Seite beginnt mit demselben Gerüst. Stufe 1 lässt zwei Zeilen weg, die große Folgen haben:

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Campus Hackathon 2026 · HTWG Konstanz</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body> ... </body>
</html>
```

- `<!DOCTYPE html>` schaltet den Standardmodus ein. Ohne ihn rendert der Browser im alten „Quirks Mode“ mit anderen Regeln für das Box-Modell.
- `lang="de"` sagt Screenreadern, in welcher Sprache sie vorlesen sollen, und hilft Übersetzern und Suchmaschinen.
- `meta viewport` sorgt dafür, dass ein Handy die Seite in seiner echten Breite darstellt. Ohne diese Zeile tut es so, als wäre es ein Desktop mit 980 Pixeln, und verkleinert alles.
- `title` erscheint im Tab, in Lesezeichen und in Suchergebnissen. „Seite“ ist kein guter Titel.

Landmarks: die Bereiche einer Seite

HTML5 hat Elemente, die Bereiche einer Seite benennen. Screenreader bieten diese Bereiche als Sprungmarken an, man nennt sie deshalb **Landmarks**.

Element	Bedeutung
`header`	Kopfbereich, meist Logo und Navigation
`nav`	Navigation, also eine Gruppe von Links zu anderen Seiten oder Bereichen
`main`	Der Hauptinhalt. Genau einmal pro Seite
`section`	Ein thematischer Abschnitt mit eigener Überschrift
`article`	Ein in sich geschlossener Inhalt, der auch allein stehen könnte (Programmpunkt, Blogpost, Karte)
`footer`	Fußbereich

Der Unterschied ist für Menschen mit Screenreader enorm. So klingt dieselbe Seite:

Div-Suppe	Semantisch
„Campus Hackathon. Programm. Anmeldung. Campus Hackathon 2026. 14. November … Name. Bearbeiten. E-Mail. Bearbeiten. Anmelden.“	„Banner. Navigation, Liste mit 2 Einträgen, Link Programm … Hauptbereich. Überschrift Ebene 1: Campus Hackathon 2026 … Formular. Name, Eingabefeld, erforderlich … Schalter Anmelden.“

> 💡 **Tipp:** Überschriften sind das Inhaltsverzeichnis eurer Seite. Genau eine `h1`, darunter `h2` für Abschnitte, darunter `h3`. Die Ebene wählt ihr nach der Struktur, nie nach der Schriftgröße. Die Größe regelt CSS.

Links und Buttons: navigieren oder handeln

In Stufe 1 sind die Navigation ein `span` mit `onclick` und der Anmelde-Knopf ein `div`. Das funktioniert mit der Maus, aber nicht mit der Tastatur, und ein Screenreader erkennt beides nicht als bedienbar.

Die Regel ist einfach:

- **Führt es woanders hin?** Dann ist es ein Link: `<a href="#programm">`.
- **Löst es eine Aktion aus?** Dann ist es ein Button: `<button type="submit">`.

Beide Elemente bringen alles mit, was man sonst mühsam nachbauen müsste: Fokus mit der Tab-Taste, Auslösen mit Enter oder Leertaste, die richtige Rolle für Screenreader.

Bilder

```
<img src="hackathon.svg" alt="Illustration: ein Browserfenster mit Code-
Klammern"
     width="600" height="240">
```

- `alt` beschreibt, was das Bild zeigt. Ist ein Bild rein dekorativ, schreibt ihr `alt=""`, dann überspringt der Screenreader es.
- `width` und `height` reservieren den Platz, bevor das Bild geladen ist. Sonst springt der Text beim Laden nach unten. In der Vorlesung zu Performance lernt ihr dafür den Fachbegriff **Layout Shift**.

Formulare

Formulare sind der Ort, an dem Nutzer mit eurer Anwendung sprechen. Ein korrektes Formular für die Anmeldung sieht so aus:

```
<form action="/anmeldung" method="post">
  <label for="name">Name</label>
```

```
  <input id="name" name="name" type="text" autocomplete="name" required>

  <label for="email">E-Mail</label>
  <input id="email" name="email" type="email" autocomplete="email" required>

  <button type="submit">Anmelden</button>
</form>
```

Jedes Attribut hat eine Aufgabe:

- `label for` und `input id` gehören zusammen. Der Screenreader liest das Label vor, und ein Klick auf das Label setzt den Fokus ins Feld.
- `name` ist der Schlüssel, unter dem der Wert an den Server geht. Ohne `name` wird das Feld nicht übertragen.
- `type="email"` gibt Handys die passende Tastatur und prüft das Format.
- `autocomplete` erlaubt dem Browser, bekannte Werte vorzuschlagen.
- `required` verhindert das Absenden leerer Pflichtfelder.

> ⚠ **Achtung:** Die Prüfung im Browser ist Komfort, keine Sicherheit. Jeder kann in den DevTools `required` löschen oder die Anfrage direkt an den Server schicken. Der Server muss jede Eingabe selbst prüfen. Das sehen wir uns in der Vorlesung zu Web Security genauer an.

Aus HTML wird der DOM

Der Browser liest das HTML und baut daraus einen Baum aus Objekten, das **Document Object Model** (DOM). Die Verschachtelung im HTML wird zur Eltern-Kind-Beziehung im Baum: `main` enthält `section`, `section` enthält `h2` und `ul`, und so weiter.

Zwei Dinge daran sind wichtig. Erstens ist der DOM das, was ihr in den DevTools unter **Elements** seht, nicht euer Quelltext. Der Browser repariert fehlerhaftes HTML stillschweigend, deshalb können beide voneinander abweichen. Zweitens ist der DOM genau die Struktur, die JavaScript in der JavaScript-Vorlesung verändert und die React in der React-Vorlesung für euch verwaltet.

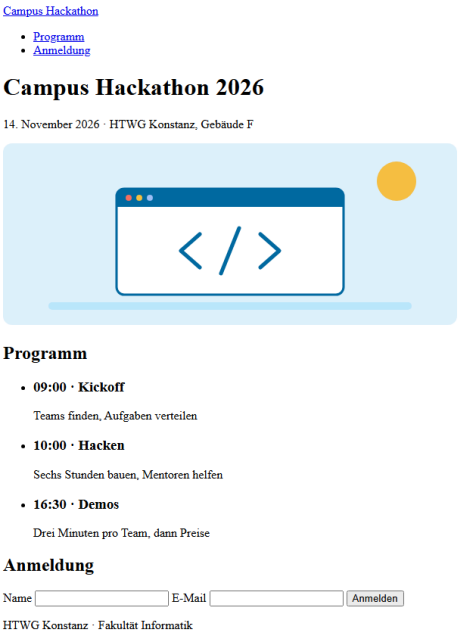


Abbildung 3: Stufe 2: semantisch korrekt, aber noch ohne CSS. Hässlich, aber richtig.

Stufe 2 sieht schlechter aus als Stufe 1, und das ist Absicht. Sie zeigt, dass die Bedeutung vollständig im HTML steckt. Alles Weitere ist Darstellung, und dafür ist CSS zuständig.

CSS beschreibt Darstellung

Stufe 3 verwendet exakt das HTML aus Stufe 2. Neu sind nur ein paar Klassen und die Datei `styles.css`. Alle Inline-Styles sind verschwunden. Wer jetzt die Hauptfarbe ändern will, ändert eine einzige Zeile.

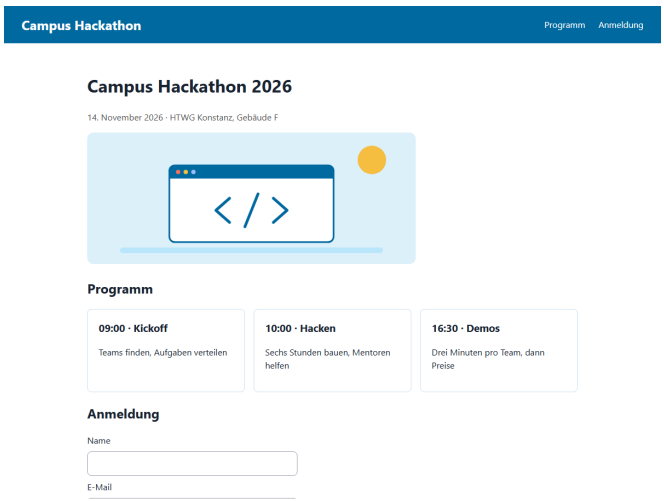


Abbildung 4: Stufe 3 auf dem Desktop

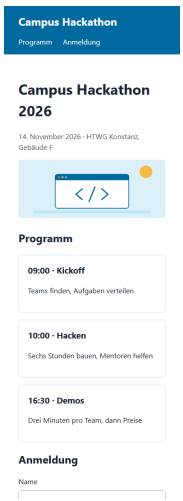


Abbildung 5: Stufe 3 auf dem Handy

Welche Regel gewinnt? Die Kaskade

Die häufigste Frage beim Arbeiten mit CSS lautet: „Warum wird meine Regel nicht angewendet?“ Die Antwort liegt fast immer in der **Kaskade**. Wenn mehrere Regeln dieselbe Eigenschaft für dasselbe Element setzen, entscheidet der Browser in dieser Reihenfolge:

1. **Herkunft und Wichtigkeit:** Eure Regeln schlagen die Standardstile des Browsers. `!important` schlägt alles, sollte aber die Ausnahme bleiben.
2. **Spezifität:** Der genauere Selektor gewinnt.
3. **Reihenfolge:** Bei gleicher Spezifität gewinnt die Regel, die später kommt.

Die Spezifität zählt ihr in drei Stellen: IDs, dann Klassen (auch Attribute und Pseudoklassen), dann Elemente.

Selektor	Spezifität (ID, Klasse, Element)
`a`	(0, 0, 1)
`.site-header a`	(0, 1, 1)
`.site-header a:hover`	(0, 2, 1)
`#anmeldung h2`	(1, 0, 1)

Im Beispiel färbt die Browser-Standardregel alle Links blau. Unsere Regel `.site-header a { color: white; }` ist spezifischer und gewinnt, deshalb sind die Links in der Kopfzeile weiß, alle anderen bleiben blau.

Dazu kommt die **Vererbung**: Manche Eigenschaften wie `color`, `font-family` und `line-height` gehen automatisch an Kindelemente über. Deshalb reicht es, die Schrift einmal am `body` zu setzen. Abstände, Rahmen und Breiten vererben sich dagegen nicht.

> 💡 **Tipp:** In den DevTools zeigt das **Styles**-Panel alle Regeln für ein Element. Regeln, die verloren haben, sind durchgestrichen. So seht ihr die Kaskade direkt.

Das Box-Modell

Jedes Element ist für den Browser eine rechteckige Box aus vier Schichten: **content** (Inhalt), **padding** (Innenabstand), **border** (Rahmen) und **margin** (Außenabstand).

Die Stolperfalle steckt in der Frage, was `width` eigentlich meint. Standardmäßig (`box-sizing: content-box`) gilt die Breite nur für den Inhalt. Padding und Rahmen kommen oben drauf. Die Programmkarten in Stufe 1 sind deshalb nicht 300 Pixel breit, sondern:

```
300 (width) + 2 × 20 (padding) + 2 × 1 (border) = 342 Pixel
```

Dazu kommen noch 2 × 10 Pixel Margin. Genau deshalb laufen die Karten auf dem Handy über den Rand. Die Lösung steht ganz oben in jedem modernen Stylesheet:

```css
*, *::before, *::after {
  box-sizing: border-box;
}
```

Mit `border-box` meint `width` die ganze Box inklusive Padding und Rahmen. Kein Kopfrechnen mehr.

Layout: Flexbox oder Grid?

Stufe 1 ordnet Elemente mit `float` an. Das war vor 15 Jahren Stand der Technik und ist heute nicht mehr nötig. Moderne Layouts bauen auf zwei Werkzeugen:

Flexbox: eine Dimension	Grid: zwei Dimensionen
Ordnet Elemente in **einer Reihe oder einer Spalte** an und verteilt den Platz dazwischen. Im Beispiel: die Kopfzeile. Logo links, Navigation rechts, beide vertikal zentriert. `.site-header { display: flex; justify-content: space-between; align-items: center; }`	Ordnet Elemente in **Zeilen und Spalten gleichzeitig** an. Im Beispiel: die Programmkarten, auf dem Desktop drei nebeneinander, alle gleich hoch. `.programm { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }`

★ Faustregel: Richtet sich das Layout nach dem **Inhalt** und läuft in eine Richtung, nehmt Flexbox. Gebt ihr ein **Raster** vor, in das sich Inhalte einfügen, nehmt Grid. Beide lassen sich verschachteln.

Responsive mit Mobile-First

Eine Seite wird auf Bildschirmen von 360 bis 2500 Pixeln Breite angezeigt. **Mobile-First** heißt: Ihr schreibt zuerst das Layout für den kleinsten Bildschirm und erweitert es dann für größere.

```css
/* Basis: eine Spalte, gilt überall */
.programm {
  display: grid;
  grid-template-columns: 1fr;
}

/* Ab 48rem Breite: drei Spalten */
@media (min-width: 48rem) {
```

```
  .programm {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Warum in dieser Richtung? Das Handy-Layout ist meist das einfachere. Man fügt für große Bildschirme etwas hinzu, statt für kleine Bildschirme Dinge wieder wegzunehmen. Dazu gehören zwei kleine Regeln, die viel Ärger ersparen: `img { max-width: 100%; }` verhindert, dass Bilder über den Rand laufen, und der Viewport-Meta-Tag aus dem Grundgerüst sorgt dafür, dass die Media Queries auf dem Handy überhaupt greifen.

Design-Token mit CSS-Variablen

Farben und Abstände wiederholen sich in jedem Stylesheet. CSS-Variablen (**Custom Properties**) geben ihnen einen Namen:

```
:root {
  --farbe-primaer: #0369a1;
  --abstand: 1rem;
}
.site-header { background: var(--farbe-primaer); }
```

Wer das Farbschema ändern oder einen Dark Mode bauen will, ändert nur noch diese Werte. Viele Design-Systeme und Frameworks arbeiten genau so.

Werkzeug: die DevTools

Die Browser-DevTools (F12) sind euer wichtigstes Werkzeug für HTML und CSS:

- **Elements:** der aktuelle DOM. Hier seht ihr, was der Browser aus eurem HTML gemacht hat.
- **Styles:** alle Regeln für das ausgewählte Element, verlorene Regeln durchgestrichen.
- **Computed** mit Box-Modell-Grafik: die tatsächlichen Maße inklusive Padding, Border und Margin.
- **Flex- und Grid-Badges** im Elements-Panel blenden die Layout-Linien ein.
- **Device Toolbar** (Strg+Umschalt+M): simuliert Handy-Bildschirme.
- **Lighthouse** prüft Accessibility und zeigt z. B. fehlende Labels oder Alt-Texte.

Was ihr davon im Projekt braucht

Im ersten Meilenstein baut ihr eine React-App mit Layout und Routing. Aus diesem Kapitel nehmt ihr mit:

- Landmarks (`header`, `nav`, `main`, `footer`) und eine saubere Überschriften-Hierarchie
- Links zum Navigieren, Buttons für Aktionen
- Formulare mit `label`, `name` und passendem `type`
- `box-sizing: border-box`, Flexbox und Grid, Mobile-First
- Farben und Abstände als CSS-Variablen

In JSX heißt `class` übrigens `className` und `for` heißt `htmlFor`. Sonst bleibt alles gleich.

Typische Fehler

⚠ **Achtung:**

- `div` oder `span` mit `onclick` statt `a` oder `button`: nicht mit der Tastatur bedienbar.
- `input` ohne `label` und ohne `name`.
- Überschriften nach Schriftgröße gewählt statt nach Struktur.
- Feste Breiten in Pixeln ohne `border-box` und ohne `max-width`.
- Viewport-Meta-Tag vergessen: Media Queries greifen auf dem Handy nicht.
- Browser-Validierung für Sicherheit halten.

Zusammenfassung

★ HTML gibt Inhalten Bedeutung, CSS gibt ihnen Gestalt. Semantisches HTML hilft Screenreadern, Tastaturnutzern, Suchmaschinen und eurem Team. In CSS entscheidet die Kaskade, welche Regel greift, das Box-Modell mit `border-box` macht Breiten vorhersagbar, Flexbox ordnet in einer Dimension, Grid in zwei, und Mobile-First baut vom kleinen zum großen Bildschirm.

Kontrollfragen

1. Eine Seite sieht auf dem Desktop einwandfrei aus. Nennt drei Gruppen von Nutzern oder Programmen, für die sie trotzdem kaputt sein kann, und woran das jeweils liegt.
2. Ein Kommilitone baut den Anmelde-Knopf als `<div class="btn" onclick="...">`. Was geht verloren, und wie sieht die richtige Lösung aus?
3. Auf ein Element wirken `a { color: blue; }` und `.site-header a { color: white; }`. Welche Farbe hat ein Link in der Kopfzeile, und warum?

4. Eine Karte hat `width: 50%`, `padding: 1rem` und einen Rahmen von 1px. Zwei Karten passen nicht nebeneinander. Erklärt die Ursache und die Lösung.
5. Ihr baut eine Kopfzeile und eine Bildergalerie. Welches Layout-Werkzeug nehmt ihr jeweils, und warum?

Zum Weiterlesen

- MDN Learn Web Development, Abschnitte HTML und CSS: developer.mozilla.org/de/docs/Learn
- Wer wissen will, woher HTML kommt (CERN 1989, Browser Wars, XHTML, Living Standard): Skript Web-Applikationen SS26, Kapitel 2, liegt als Zusatzmaterial in Moodle.